

Dokumentacja projektu
wykonywanego w ramach zajęć
BAZY DANYCH II

Analizator Architektury



AGH

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

09.06.2026

Spis treści

1	Projekt koncepcji i założenia	3
1.1	Cel systemu	3
1.2	Założenia funkcjonalne	3
1.3	Założenia нефunkcjonalne i technologiczne	3
1.4	Koncepcja architektoniczna	4
2	Architektura i przepływ danych	4
2.1	Ogólna architektura i przepływ danych	4
2.2	Sekwencja operacji: Analiza strefy wpływu (Blast Radius)	5
3	Projekt logiczny	6
3.1	Struktura grafu i implementacja poleceń bazodanowych	6
3.2	Model logiczny grafu	6
3.2.1	Węzły (Labels) i ich właściwości	6
3.2.2	Relacje między węzłami	7
3.3	Implementacja poleceń realizujących założenia funkcjonalne	7
3.3.1	Tworzenie infrastruktury i zarządzanie projektami	8
3.3.2	Algorytm wyznaczania strefy wpływu awarii (Blast Radius)	8
3.3.3	Kontekst organizacyjny i zarządzanie klastrami	8
4	Projekt funkcjonalny i User Experience (UX)	10
4.1	Koncepcja wizualna i ergonomia interfejsu	10
4.2	Przepływ pracy (User Flow) i główne widoki	11
4.2.1	Autoryzacja i zarządzanie projektami	11
4.2.2	Główny obszar roboczy (Canvas) i nawigacja	12
4.3	Interakcja z grafem i panele kontekstowe	12
4.4	Wizualizacja analizy Blast Radius	13
4.5	Podsumowanie aspektów UX	14
5	Opracowanie dokumentacji technicznej	14
5.1	Dokumentacja kodu źródłowego (Javadoc)	14
5.2	Dokumentacja interfejsu API (REST)	15
5.3	Dokumentacja wdrożeniowa i środowiskowa	16
6	Źródła	17

1 Projekt koncepcji i założenia

Rozdział ten prezentuje ogólną koncepcję proponowanego rozwiązania oraz definiuje kluczowe założenia projektowe, zarówno w warstwie funkcjonalnej, jak i нефункциональной.

1.1 Cel systemu

Głównym celem projektu jest stworzenie interaktywnego narzędzia wspomagającego pracę architektów oprogramowania oraz inżynierów DevOps. System ma umożliwiać wizualizację ekosystemu mikrousług, śledzenie własności (odpowiedzialności) poszczególnych zespołów programistycznych oraz analizę strefy wpływu (tzw. *blast radius*) w przypadku potencjalnych awarii. Narzędzie ma pomóc w priorytetyzacji napraw i komunikacji ryzyka poprzez przejrzyste mapowanie zależności.

1.2 Założenia funkcjonalne

Na podstawie analizy potrzeb określono następujący zestaw wymagań funkcjonalnych, które system musi realizować:

- **Zarządzanie architekturą:** Użytkownik musi mieć możliwość dodawania węzłów (reprezentujących mikrousługi, bazy danych, systemy zewnętrzne) oraz definiowania połączeń (zależności) między nimi.
- **Analiza strefy wpływu (Blast Radius):** System musi potrafić wyznaczyć i wyróżnić wizualnie wszystkie usługi, które są bezpośrednio lub pośrednio zależne od wybranego, awaryjnego węzła.
- **Interaktywna wizualizacja:** Topologia systemu musi być renderowana w postaci interaktywnego grafu, umożliwiającego przesuwanie węzłów, grupowanie ich w klastry oraz przypinanie notatek dokumentacyjnych.
- **Kontekst organizacyjny:** Możliwość rejestracji zespołów programistycznych, dodawania pracowników oraz przypisywania zespołów jako opiekunów (maintainerów) konkretnych mikrousług.
- **Izolacja danych (Multi-tenancy):** Każdy użytkownik musi posiadać własną, odizolowaną przestrzeń roboczą (projekty). Dane jednego użytkownika nie mogą być widoczne dla innych.

1.3 Założenia нефункциональные i technologiczne

Aby zapewnić wysoką wydajność, bezpieczeństwo oraz skalowalność rozwiązania, przyjęto następujące założenia technologiczne:

- **Modelowanie danych:** Ze względu na sieciowy charakter architektury mikrousług, dane będą przechowywane i przetwarzane przy użyciu grafowej bazy

danych (Neo4j). Pozwoli to na zoptymalizowanie zapytań o ścieżki zależności i analizę wpływu.

- **Bezpieczeństwo i uwierzytelnianie:** Dostęp do systemu będzie chroniony za pomocą mechanizmu JWT (JSON Web Token). Hasła użytkowników będą przechowywane w postaci bezpiecznych hashów.
- **Architektura klient-serwer:** Aplikacja zostanie podzielona na warstwę serwerową (backend) udostępniającą interfejs API w stylu REST oraz warstwę kliencką (frontend) działającą w przeglądarce internetowej.
- **Stos technologiczny:**
 - *Backend:* Java 21, framework Spring Boot 3.
 - *Frontend:* HTML5, Vanilla JavaScript, framework CSS Tailwind oraz biblioteka Vis.js do renderowania grafów sieciowych.
 - *Baza danych:* Neo4j w wersji 5.
- **Wydajność interfejsu:** Wizualizacja powinna działać płynnie nawet przy grafach zawierających kilkadziesiąt węzłów i krawędzi, wykorzystując silnik fizyczny do automatycznego układania elementów.

1.4 Koncepcja architektoniczna

Proponowane rozwiązanie opiera się na scentralizowanym serwerze aplikacyjnym, który komunikuje się z grafową bazą danych przy użyciu języka zapytań Cypher. Warstwa prezentacji pobiera stan grafu poprzez żądania HTTP i dynamicznie buduje model wizualny w przeglądarce użytkownika. Dzięki zastosowaniu bazy Neo4j, operacje takie jak obliczanie najkrótszej ścieżki czy trawersowanie drzewa zależności (niezbędne do wyznaczenia *blast radius*) są realizowane natywnie po stronie bazy danych, co znacząco odciąża główny serwer aplikacyjny.

2 Architektura i przepływ danych

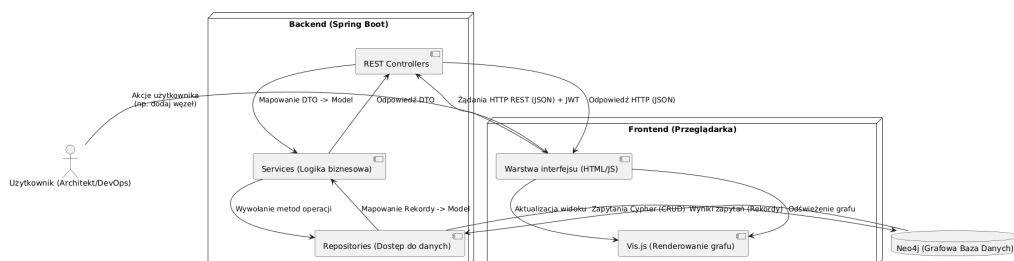
W niniejszym rozdziale zaprezentowano architekturę systemu ze szczególnym uwzględnieniem przepływu informacji między jego poszczególnymi warstwami. Zastosowano diagramy UML w celu zobrazowania struktury komponentów oraz sekwencji wywołań dla kluczowych procesów biznesowych.

2.1 Ogólna architektura i przepływ danych

System został zaprojektowany w oparciu o architekturę trójwarstwową, w której wyraźnie oddzielono warstwę prezentacji (frontend), warstwę logiki biznesowej (backend) oraz warstwę trwałego składowania danych (baza grafowa).

Jak przedstawiono na diagramie komponentów (Rysunek 1), interakcja rozpoczyna się od akcji użytkownika w interfejsie graficznym. Aplikacja kliencka, wykorzystująca bibliotekę Vis.js, zarządza stanem wizualnym grafu. Wszelkie operacje modyfikujące stan systemu (np. dodanie nowego węzła lub relacji) są mapowane na żądania HTTP REST i przesyłane do serwera Spring Boot.

W warstwie backendu żądania są odbierane przez kontrolery (REST Controllers), które następnie przekazują przetwarzanie do odpowiednich serwisów (Services). Serwisy realizują logikę biznesową i komunikują się z repozytoriami (Repositories), których zadaniem jest translacja wywołań obiektowych na natywne zapytania w języku Cypher. Baza danych Neo4j przetwarza zapytania i zwraca wyniki, które po zmapowaniu na obiekty transferu danych (DTO) wracają tą samą ścieżką do aplikacji klienckiej, powodując odświeżenie interfejsu.



Rysunek 1: Diagram komponentów i przepływu danych w systemie

2.2 Sekwencja operacji: Analiza strefy wpływu (Blast Radius)

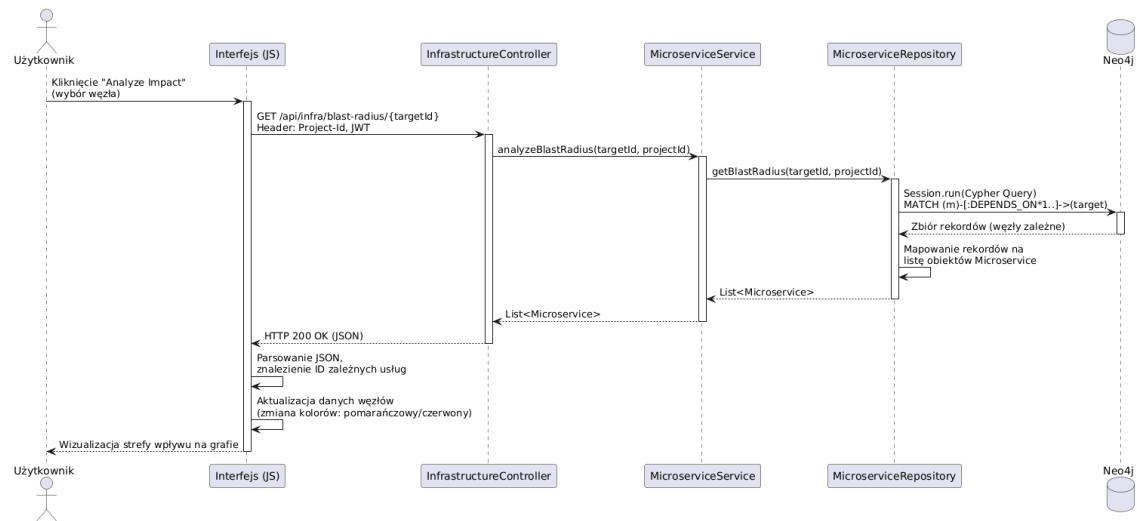
Jedną z najważniejszych funkcji systemu jest analiza strefy wpływu awarii. Proces ten wymaga głębokiej analizy grafu zależności, co zostało zrealizowane z wykorzystaniem możliwości bazy Neo4j. Diagram sekwencji (Rysunek 2) szczegółowo ilustruje kroki realizowane podczas wywołania tej funkcji.

Gdy użytkownik wybierze konkretny węzeł (mikrouslugę) i zażąda analizy wpływu, aplikacja kliencka wysyła asynchroniczne żądanie `GET` do punktu końcowego `/api/infra/blast-radius/{targetId}`. Wraz z żądaniem przesyłany jest token JWT oraz identyfikator projektu w nagłówkach HTTP.

Kontroler `InfrastructureController` przyjmuje żądanie i deleguje zadanie do serwisu `MicroserviceService`, który z kolei wywołuje metodę w repozytorium `MicroserviceRepository`. Repozytorium inicjuje sesję z bazą Neo4j i wykonuje zoptymalizowane zapytanie grafowe: `MATCH (m)-[:DEPENDS_ON*1..]->(target)`, które rekurencyjnie wyszukuje wszystkie węzły zależne od wskazanego celu.

Po przetworzeniu zapytania przez silnik bazy danych, zwrócone rekordy są mapowane na listę obiektów domenowych, a ostatecznie na format JSON. Po otrzymaniu odpowiedzi interfejs użytkownika (JS) parsuje dane i dynamicznie aktualizuje właściwości węzłów na płótnie Vis.js — węzeł awaryjny otrzymuje kolor czerwony i powiększony rozmiar, natomiast wszystkie usługi od niego zależne oznaczane są

kolorem pomarańczowym.



Rysunek 2: Diagram sekwencji dla procesu analizy strefy wpływu (Blast Radius)

3 Projekt logiczny

3.1 Struktura grafu i implementacja poleceń bazodanowych

W celu precyzyjnego odwzorowania topologii sieci mikrousług oraz powiązań organizacyjnych, w projekcie zastosowano grafową bazę danych Neo4j. Rozdział ten zawiera szczegółowy opis struktury opracowanego modelu grafowego, definicje poszczególnych węzłów i relacji, a także zestawienie zapytań w języku Cypher, które bezpośrednio realizują założenia funkcjonalne systemu.

3.2 Model logiczny grafu

Baza danych przechowuje informacje w postaci struktury grafowej, w której wyróżniono siedem głównych typów węzłów (etykiet) oraz relacje zachodzące pomiędzy nimi. Taki model pozwala na elastyczne zarządzanie powiązaniami i natywne przeszukiwanie głębokich struktur zależności.

3.2.1 Węzły (Labels) i ich właściwości

W strukturze grafu zdefiniowano następujące węzły:

- **User** – reprezentuje zarejestrowanego użytkownika systemu. Posiada właściwości: `id`, `username` oraz `password` (przechowywane w postaci zahashowanej).
- **Project** – przestrzeń robocza (projekt) należąca do danego użytkownika, zapewniająca izolację danych. Posiada właściwości: `id`, `name` oraz `ownerId`.

- **Microservice** – podstawowy element infrastruktury (usługa, baza danych, cache). Posiada właściwości: `id`, `name` (etykieta komponentu) oraz `language` (technologia wykonania) .
- **Team** – zespół deweloperski odpowiedzialny za utrzymanie usług. Posiada właściwości: `id` oraz `name`.
- **Worker** – pracownik (inżynier) przypisany do konkretnego zespołu. Posiada właściwości: `id`, `name` oraz `role` (np. Senior DevOps) .
- **Cluster** – logiczna grupa powiązanych mikrousług (np. domena biznesowa). Posiada właściwości: `id`, `name` oraz `color` (zapisany w formacie HEX).
- **Note** – notatka dokumentacyjna przypięta do obszaru roboczego, usługi lub klastra. Posiada właściwości: `id`, `title`, `text` oraz `color`.

3.2.2 Relacje między węzłami

Powiązania pomiędzy obiektami w systemie są ściśle zdefiniowane za pomocą ukierunkowanych relacji:

- `(:Project)-[:OWNED_BY]->(:User)` – określa prawo własności do projektu .
- `(:Microservice)-[:IN_PROJECT]->(:Project)`, `(:Team)-[:IN_PROJECT]->(:Project)`, `(:Cluster)-[:IN_PROJECT]->(:Project)` – zapewniają pełną wielodostępność (multi-tenancy) poprzez zakotwiczenie elementów infrastruktury i organizacji w konkretnym projekcie .
- `(:Microservice)-[:DEPENDS_ON]->(:Microservice)` – kluczowa relacja techniczna określająca, że jedna usługa wymaga do działania innej usługi (np. API Gateway zależy od Auth Service) .
- `(:Microservice)-[:MAINTAINED_BY]->(:Team)` – wiąże warstwę techniczną z organizacyjną, wskazując zespół odpowiedzialny za komponent .
- `(:Worker)-[:WORKS_IN]->(:Team)` – definiuje przynależność inżyniera do struktury zespołowej [?].
- `(:Microservice)-[:BELONGS_TO]->(:Cluster)` – przypisuje mikrousługę do określonego klastra (domeny).
- `(:Note)-[:RELATES_TO]->(:Microservice)` lub `(:Note)-[:BELONGS_TO]->(:Cluster)` – reprezentuje powiązanie dokumentacji z obiektem technicznym.

3.3 Implementacja poleceń realizujących założenia funkcjonalne

Medyczna i techniczna skuteczność operacji na grafie została zapewniona poprzez implementację dedykowanych zapytań Cypher w warstwie repozytoriów aplikacji Spring Boot. Poniżej przedstawiono kluczowe zapytania odpowiadające wymaganiom funkcjonalnym projektu.

3.3.1 Tworzenie infrastruktury i zarządzanie projektami

Podczas tworzenia nowego projektu, system wiąże go z zalogowanym użytkownikiem systemu przy użyciu następującego zapytania:

Listing 1: Zapytanie Cypher tworzące projekt

```
MATCH (u:User {username: $username})
CREATE (p:Project {id: randomUUID(), name: $name, ownerId: $username})
CREATE (p)-[:OWNED_BY]->(u)
RETURN p.id as id, p.name as name, p.ownerId as ownerId
```

Rejestracja nowej mikrousługi (węzła infrastruktury) wymaga jednoczesnego powiązania jej z projektem, co zabezpiecza system przed wyciekiem danych pomiędzy użytkownikami [?]:

Listing 2: Rejestracja mikrousługi w projekcie

```
MATCH (p:Project {id: $projectId})
MERGE (m:Microservice {id: $id})
SET m.name = $name, m.language = $language
MERGE (m)-[:IN_PROJECT]->(p)
```

Definiowanie technicznych połączeń (krawędzi grafu) między mikrousługami realizowane jest poprzez weryfikację, czy oba komponenty należą do tego samego projektu, a następnie utworzenie relacji skierowanej :

Listing 3: Tworzenie relacji zależności

```
MATCH (source {id: $sourceId})-[:IN_PROJECT]->(:Project {id: $projectId}),
      (target {id: $targetId})-[:IN_PROJECT]->(:Project {id: $projectId})
MERGE (source)-[:DEPENDS_ON]->(target)
```

3.3.2 Algorytm wyznaczania strefy wpływu awarii (Blast Radius)

Najbardziej krytyczną funkcjonalnością systemu jest rekurencyjne przeszukiwanie grafu w celu określenia, które komponenty ucierpią w wyniku awarii wybranej usługi. Dzięki zastosowaniu bazy grafowej, operacja ta sprowadza się do jednego, niezwykle wydajnego zapytania z użyciem operatora ścieżki o zmiennej długości (`[*1..]`) :

Listing 4: Trawersowanie grafu w celu wyznaczenia blast radius

```
MATCH (m:Microservice)-[:IN_PROJECT]->(:Project {id: $projectId})
MATCH (m)-[:DEPENDS_ON*1..]->(target {id: $targetId})
RETURN DISTINCT m.id AS id, m.name AS name, m.language AS language
```

Zasada działania: Zapytanie filtruje mikrousługi w ramach wybranego projektu, a następnie odnajduje wszystkie węzły `m`, od których istnieje dowolnie długa ścieżka relacji `:DEPENDS_ON` prowadząca do uszkodzonego węzła `target`. Wynik zwraca unikalną listę usług wyższego poziomu (upstream), które zostaną dotknięte awarią kaskadową.

3.3.3 Kontekst organizacyjny i zarządzanie klastrami

Przypisanie odpowiedzialności za dany komponent techniczny do zespołu programistycznego realizowane jest za pomocą relacji `:MAINTAINED_BY` :

Listing 5: Powiązanie mikrousługi z zespołem

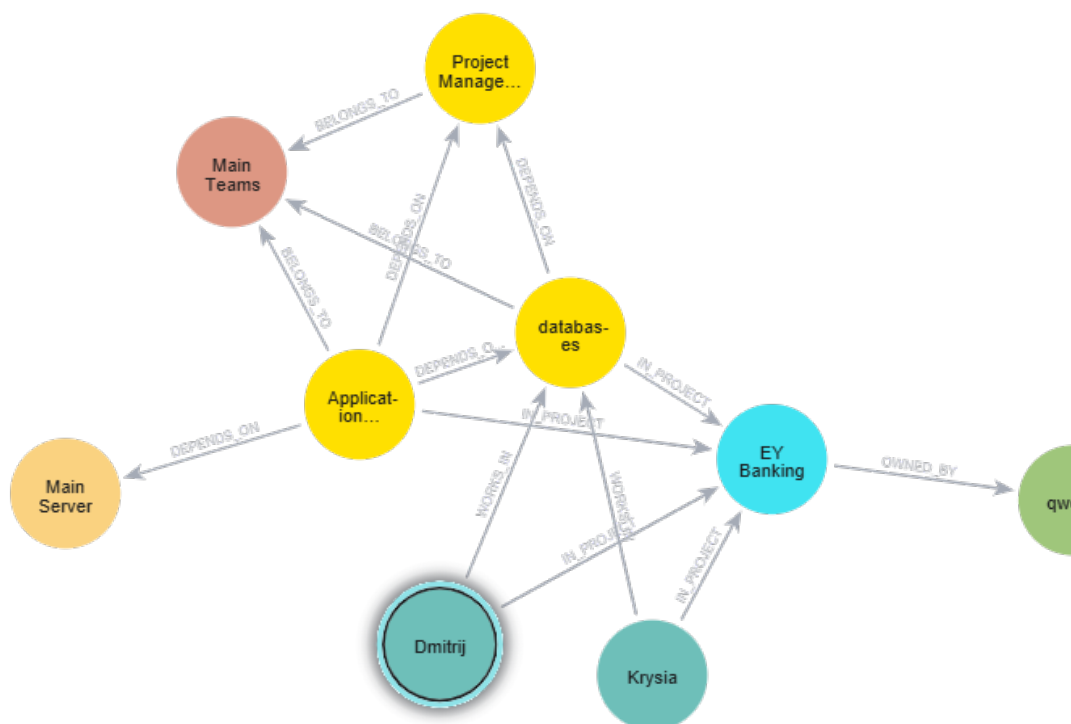
```
MATCH (m:Microservice {id: $serviceId})-[:IN_PROJECT]->(:Project {id: $projectId})
MATCH (t:Team)-[:IN_PROJECT]->(:Project {id: $projectId})
WHERE toString(t.id) = $teamId OR toString(id(t)) = $teamId
MERGE (m)-[:MAINTAINED_BY]->(t)
```

Podczas grupowania węzłów w logiczne klastry (architektura domenowa), system tworzy nowy węzeł klastra i dynamicznie przepina wskazane mikrousługi, usuwając ich ewentualne poprzednie powiązania klastrowe :

Listing 6: Grupowanie węzłów w klastry

```
MATCH (p:Project {id: $projectId})
MERGE (c:Cluster {id: randomUUID()})
SET c.name = $name, c.color = $color
MERGE (c)-[:IN_PROJECT]->(p)
WITH c
UNWIND $nodeIds AS nodeId
MATCH (m {id: nodeId})-[:IN_PROJECT]->(:Project {id: $projectId})
OPTIONAL MATCH (m)-[:old:BELONGS_TO]->(:Cluster)
DELETE old
MERGE (m)-[:BELONGS_TO]->(c)
```

Dzięki tak zaprojektowanej strukturze zapytań i powiązań, warstwa backendowa aplikacji w pełni i wydajnie realizuje wszystkie założone cele biznesowe, zachowując niski koszt obliczeniowy operacji grafowych.



Rysunek 3: Przykładowy graf

4 Projekt funkcjonalny i User Experience (UX)

W niniejszym rozdziale omówiono projekt funkcjonalny aplikacji ze szczególnym uwzględnieniem doświadczeń użytkownika (User Experience) oraz interfejsu graficznego (User Interface). Głównym celem projektowym było stworzenie intuicyjnego, nowoczesnego i responsywnego środowiska pracy dla architektów oprogramowania i inżynierów DevOps, które ułatwia zarządzanie skomplikowanymi topologiami mikrosług.

4.1 Koncepcja wizualna i ergonomia interfejsu

Interfejs użytkownika został zaprojektowany w oparciu o ciemny motyw (Dark Mode), co jest standardem w nowoczesnych narzędziach deweloperskich (IDE, terminale, systemy monitoringu). Ciemne tło redukuje zmęczenie wzroku podczas długotrwałej pracy, a jednocześnie pozwala na wyraźne wyeksponowanie kolorowych węzłów

grafu i powiadomień.

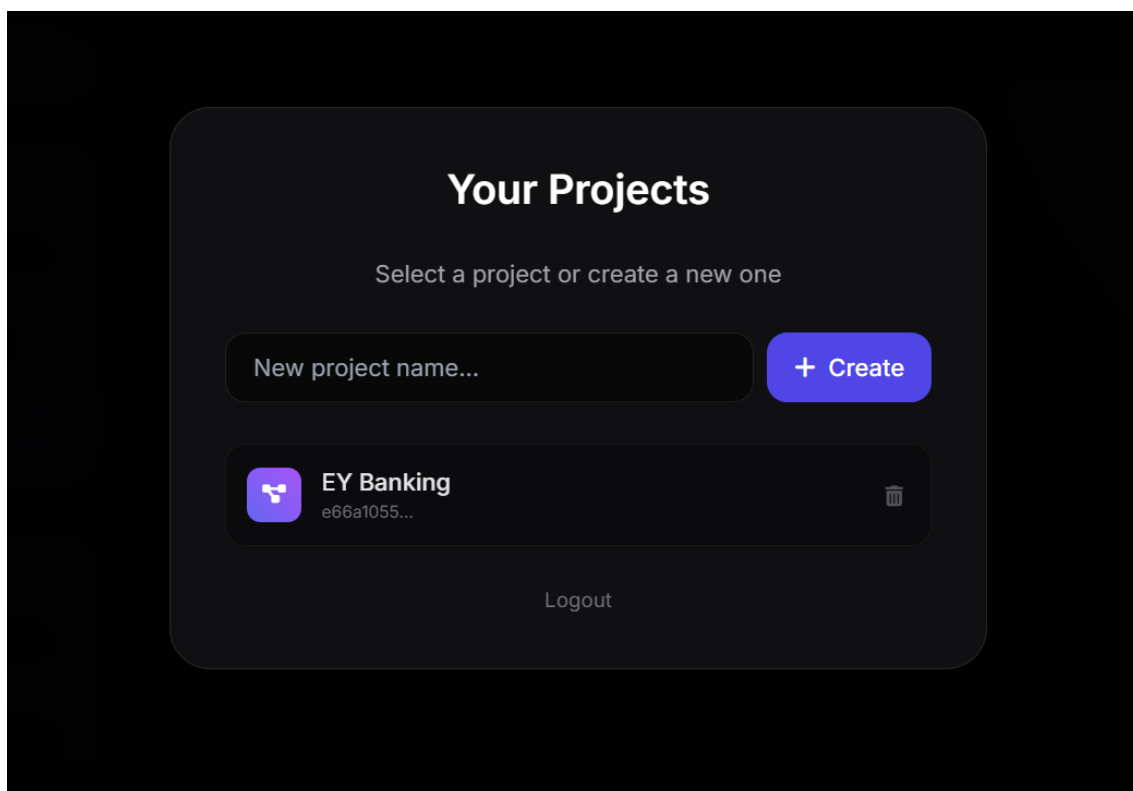
Wykorzystano styl architektoniczny *Glassmorphism* (efekt matowego szkła) przy użyciu frameworka Tailwind CSS. Panele nawigacyjne, okna dialogowe oraz menu kontekstowe posiadają delikatną przezroczystość i rozmycie tła (backdrop-blur), co nadaje aplikacji wrażenie głębi i nowoczesności, nie odciągając uwagi od głównego elementu, jakim jest płótno grafu.

4.2 Przepływ pracy (User Flow) i główne widoki

Ścieżka użytkownika w systemie została zaprojektowana w taki sposób, aby minimalizować liczbę kliknięć potrzebnych do wykonania kluczowych akcji.

4.2.1 Autoryzacja i zarządzanie projektami

Pierwszym punktem styku użytkownika z aplikacją jest ekran logowania/rejestracji. Został on zrealizowany w formie wyśrodkowanego, modalnego okna. Po pomyślnym uwierzytelnieniu użytkownik przechodzi do widoku wyboru projektu, gdzie może kontynuować pracę nad istniejącą architekturą lub utworzyć nową przestrzeń roboczą.

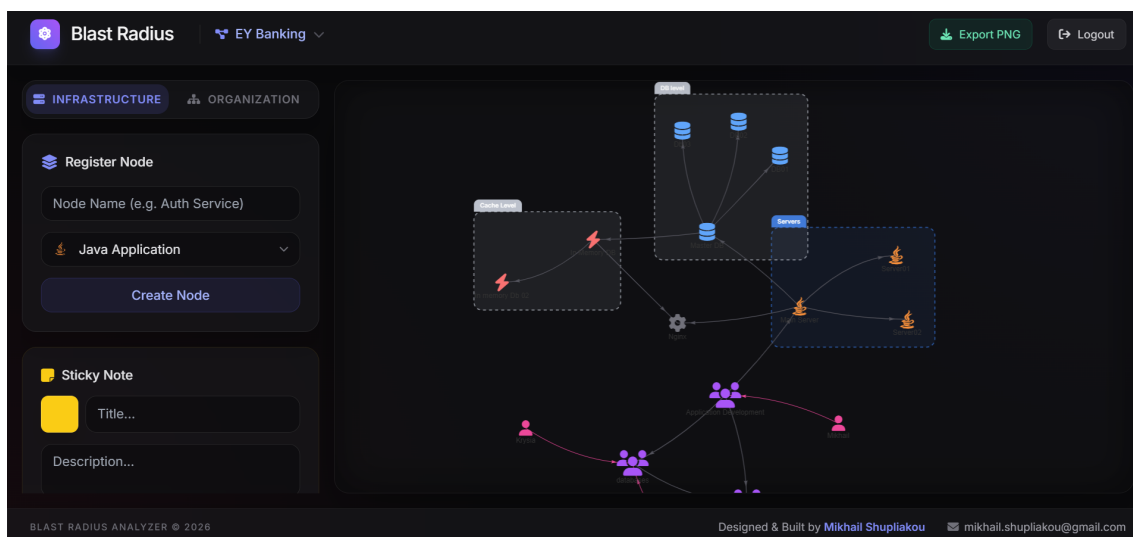


Rysunek 4: Widok wyboru przestrzeni roboczej (projektu)

4.2.2 Główny obszar roboczy (Canvas) i nawigacja

Po wyborze projektu użytkownik przenoszony jest do głównego obszaru roboczego. Interfejs został podzielony na trzy kluczowe strefy:

- **Pasek górny (Header):** Zawiera informacje o bieżącym projekcie, przycisk eksportu grafu do formatu PNG oraz opcję wylogowania.
- **Panel boczny (Sidebar):** Znajduje się po lewej stronie i został podzielony na zakładki (Infrastruktura i Organizacja). Służy jako centrum sterowania do dodawania nowych węzłów, klastrów, notatek oraz inicjowania analizy strefy wpływu.
- **Płótno grafu (Canvas):** Centralny, największy element interfejsu (wykorzystujący Vis.js), na którym renderowana jest interaktywna topologia systemu.



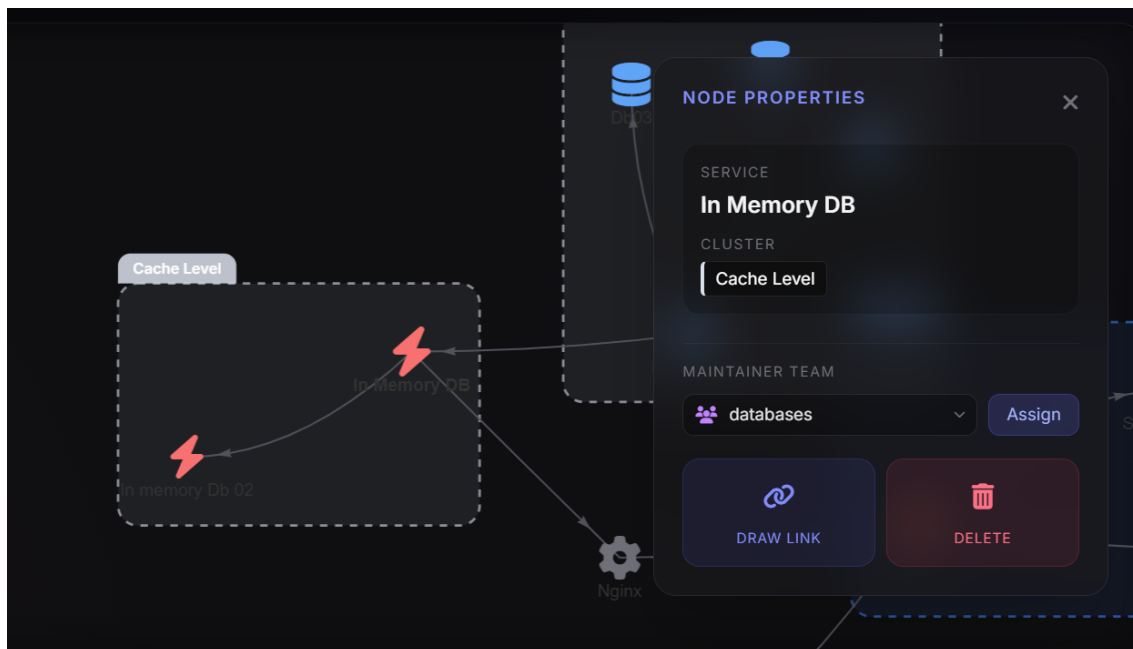
Rysunek 5: Główny obszar roboczy z panelem bocznym i interaktywnym grafem

4.3 Interakcja z grafem i panele kontekstowe

Doświadczenie użytkownika (UX) opiera się w dużej mierze na bezpośredniej interakcji z elementami grafu. Węzły posiadają dedykowane ikony reprezentujące różne technologie (np. Java, Python, bazy danych PostgreSQL, Redis), co ułatwia błyskawiczne odczytywanie architektury bez konieczności sprawdzania szczegółów każdego elementu.

Zaznaczenie pojedynczego węzła aktywuje pływający panel kontekstowy w prawym górnym rogu ekranu (*Action Panel*). Panel ten dynamicznie dostosowuje swoją zawartość do wybranego typu obiektu — dla mikrousługi wyświetla jej nazwę, technologię, przypisany klaster oraz pozwala na powiązanie z zespołem utrzymaniowym, wejście w tryb rysowania relacji (łączenia) lub usunięcie węzła.

Zaznaczenie wielu węzłów jednocześnie udostępnia opcję grupowania ich w nowy, nazwany klaster (tzw. architekturę domenową). Klastry są wizualizowane jako obwiednie z przezroczystym wypełnieniem, co ułatwia organizację przestrzenną złożonych systemów.



Rysunek 6: Pływający panel akcji kontekstowych dla wybranego węzła

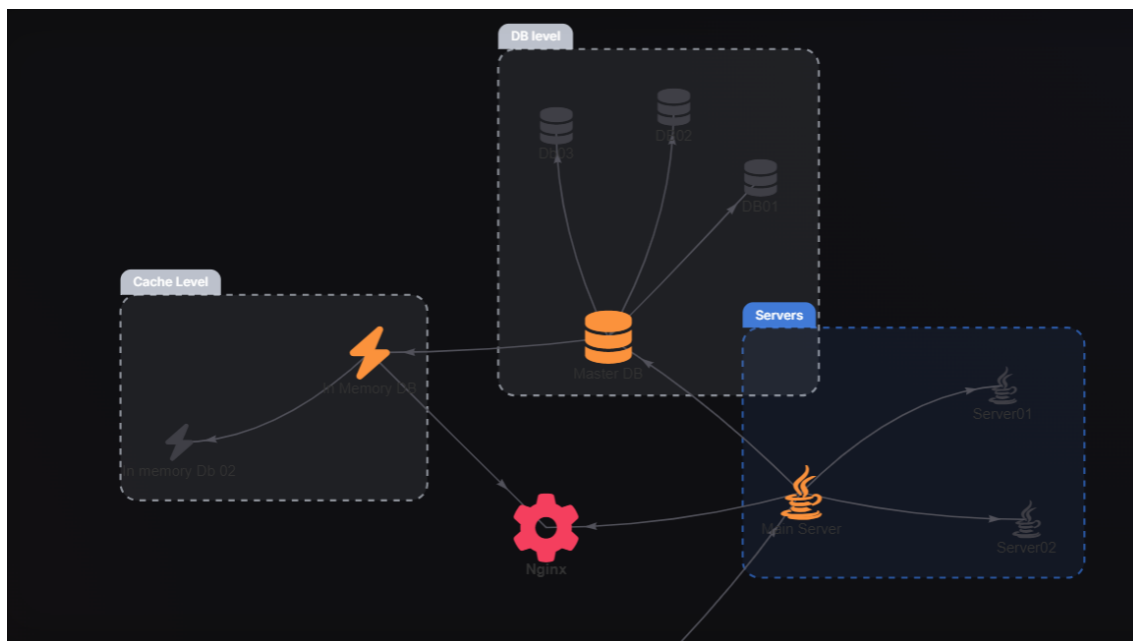
4.4 Wizualizacja analizy Blast Radius

Najważniejszą funkcją analityczną systemu jest symulacja strefy wpływu awarii. UX tego procesu zaprojektowano z naciskiem na maksymalną czytelność informacji zwrotnej.

Po wskazaniu docelowego węzła w lewym panelu bocznym i uruchomieniu analizy, system zmienia wizualne właściwości odpowiednich elementów na grafie:

- **Węzeł awaryjny (Target):** Zmienia kolor na jaskrawoczerwony (#f43f5e) i znacznie powiększa swój rozmiar, sygnalizując epicentrum problemu.
- **Węzły zależne (Affected):** Wszystkie usługi, które ucierpią w wyniku awarii (bezpośrednio i pośrednio), zmieniają kolor na pomarańczowy (#fb923c) i również ulegają powiększeniu.
- **Węzły bezpieczne:** Elementy nienarażone na awarię zachowują swój neutralny, szary kolor, stając się tłem dla zaistniałego problemu.

Takie podejście pozwala na natychmiastowe zidentyfikowanie ryzyka przez architekta, bez konieczności czytania tabelarycznych raportów czy ręcznego śledzenia linii powiązań.



Rysunek 7: Wizualna reprezentacja analizy strefy wpływu (Blast Radius)

4.5 Podsumowanie aspektów UX

Zastosowanie interaktywnego, ukierunkowanego na fizykę silnika graficznego (Vis.js) w połączeniu z minimalistycznym designem paneli bocznych sprawia, że aplikacja jest przystępna nawet przy analizie dużych i skomplikowanych ekosystemów. Dodatkowe ułatwienia, takie jak automatyczne zwijanie list wyboru (custom selects) z kategoryzacją technologii czy powiadomienia typu *toast* podczas łączenia węzłów, znacząco podnoszą komfort pracy użytkownika.

5 Opracowanie dokumentacji technicznej

Rozdział ten opisuje podejście do tworzenia dokumentacji technicznej projektu, która stanowi kluczowy element zapewniający łatwość utrzymania, skalowania oraz wdrażania nowych programistów do zespołu. Dokumentacja została podzielona na trzy główne obszary: dokumentację kodu źródłowego, dokumentację interfejsu API oraz instrukcje wdrożeniowe.

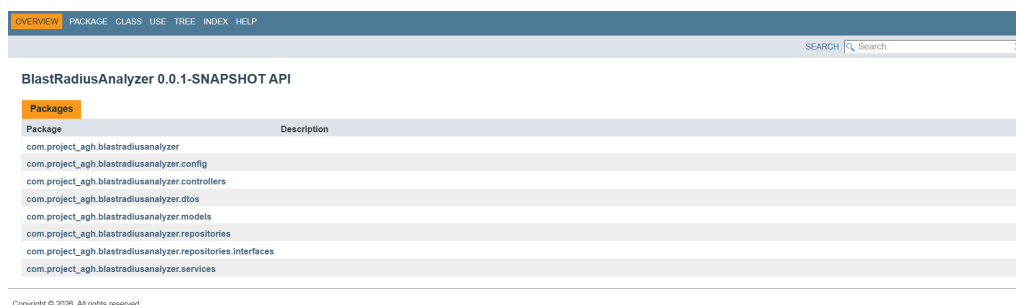
5.1 Dokumentacja kodu źródłowego (Javadoc)

Do udokumentowania warstwy backendowej, napisanej w języku Java i opartej na frameworku Spring Boot, wykorzystano standard *Javadoc*. Znaczniki dokumentacyjne zostały umieszczone bezpośrednio nad deklaracjami klas, interfejsów oraz kluczowych metod, opisując ich przeznaczenie, przyjmowane parametry (`@param`) oraz zwracane wartości (`@return`).

Proces generowania dokumentacji został zautomatyzowany przy użyciu narzędzia budującego Maven. Po wywołaniu odpowiedniego polecenia w terminalu (`mvnw javadoc:javadoc`), system automatycznie analizuje kod źródłowy i generuje statyczną witrynę w formacie HTML. Wygenerowane pliki trafiają do katalogu `target/site/apidocs`, skąd mogą być łatwo przeglądane w dowolnej przeglądarce internetowej.

Wygenerowana dokumentacja obejmuje najważniejsze warstwy systemu:

- **Modele i DTO (Data Transfer Objects):** Opis struktury danych przesyłanych między warstwami i mapowanych na węzły grafu.
- **Repozytoria:** Dokumentacja niestandardowych operacji bazodanowych i wywołań w języku Cypher.
- **Serwisy:** Opis logiki biznesowej, takiej jak algorytmy analizy *Blast Radius*.



Rysunek 8: Wygenerowana strona główna dokumentacji Javadoc dla klas projektu

5.2 Dokumentacja interfejsu API (REST)

Aplikacja opiera się na architekturze klient-serwer, w której komunikacja odbywa się za pośrednictwem zapytań HTTP REST. Struktura żądań i odpowiedzi została ustandaryzowana z wykorzystaniem obiektów w formacie JSON. W dokumentacji projektowej wyodrębniono główne kontrolery (tzw. *endpoints*):

- `/api/auth/*` – obsługa uwierzytelniania, rejestracji i generowania tokenów dostępu (JWT).
- `/api/infra/*` – punkt dostępowy do zarządzania topologią infrastruktury: węzłami, klastrami, relacjami zależności oraz wyliczaniem strefy wpływu awarii.
- `/api/org/*` – operacje związane ze strukturą organizacyjną (tworzenie zespołów, przypisywanie pracowników).

Dla punktów końcowych wymagających autoryzacji zdefiniowano wymóg przekazywania nagłówków HTTP: `Authorization` (z tokenem typu Bearer) oraz `Project-Id` w celu izolacji danych użytkownika.

5.3 Dokumentacja wdrożeniowa i środowiskowa

Aby ułatwić uruchomienie aplikacji w środowisku lokalnym, ewaluacyjnym lub produkcyjnym, w głównym katalogu repozytorium przygotowano plik konfiguracyjny `README.md`. Zawiera on zwięzłą instrukcję krok po kroku, obejmującą:

1. Wymagania wstępne (np. Java 21, Maven, zainstalowane środowisko Docker).
2. Procedurę przygotowania bazy danych Neo4j. W tym celu opracowano plik `docker-compose.yml`, który pozwala na błyskawiczne podniesienie kontenera z bazą grafową, automatycznie definiując zmienne środowiskowe, takie jak hasło administracyjne (`NEO4J_AUTH`).
3. Instrukcję budowania projektu oraz uruchamiania serwera aplikacyjnego Spring Boot przy użyciu skryptu *Maven Wrapper*.

Dzięki ustandaryzowaniu dokumentacji wdrożeniowej przy wykorzystaniu konteneryzacji, proces uruchamiania systemu przez nowych inżynierów lub architektów został maksymalnie uproszczony i zautomatyzowany.

6 Źródła

1. [Neo4j](#) - Grafowa baza danych wykorzystana w projekcie do analizy zależności
2. [Spring Boot](#) - Framework użyty do budowy warstwy serwerowej (backendu)
3. [Vis.js](#) - Biblioteka JavaScript do interaktywnej wizualizacji topologii i grafów
4. [Tailwind CSS](#) - Framework CSS wykorzystany do stworzenia nowoczesnego interfejsu użytkownika
5. [Gotowy projekt Blast Radius Analityzer na platformie GitHub](#)